Algorithmique — exemples progressifs

Chaque chapitre correspond à un niveau ou à une étape. Choisissez son titre dans le sommaire pour y accéder directement.

Sommaire

1 - Collège 4
1.1 - Ce que ce document couvre 4
1.2 - Variables et sortie 4
1.3 - Une valeur initialisée 4
1.4 - Une boucle simple 4
1.5 - Une répétition sans compteur 4
1.6 - Une condition simple 5
1.7 - Une boucle avec un pas négatif 5
1.8 - Une boucle qui filtre 5
1.8.1 - Les multiples de 3 entre 1 et 20 5
1.9 - Chercher les diviseurs d'un entier 6
1.10 - Filtrer par intervalle 6
1.11 - L'opérateur ternaire 7
1.12 - Conditions composées 7
1.13 - L'accumulateur 7
1.14 - Compter sous condition 7
1.15 - Les boucles conditionnelles 8
1.16 - Interrompre une répétition 8
2 - Lycée 9
2.1 - Trois conteneurs, une seule grammaire 9
2.2 - La liste 9
2.2.1 - Grandir par concaténation 9
2.3 - Le dictionnaire 9
2.4 - La matrice 10
2.5 - Composer les conteneurs 10
2.6 - Les quatre types de nombres 10
2.7 - Le type protège 10
2.8 - Mesurer, chercher, ranger 11
2.8.1 - La division euclidienne 11
2.8.2 - La chaîne de caractères 12
2.8.3 - Écrire une suite sur une ligne 12
2.8.4 - La tranche 13
2.8.5 - Le classement des chaînes 13
2.9 - Deux sortes de fonctions 13
2.10 - La forme la plus simple 13
2.11 - Un corps entre accolades 13
2.11.1 - Plusieurs instructions 14
2.12 - Fonctions et boucles 14
2.13 - Recevoir et rendre un conteneur 14
2.13.1 - Rendre une collection 15

2.13.2 - Chercher dans une collection reçue 15
2.13.3 - Modifier une copie locale 15
2.13.4 - Les chaînes de caractères 15
2.14 - Les primitives dans un corps 15
2.15 - Les k plus proches voisins 15
3 - Enseignement supérieur 18
3.1 - La récursivité 18
3.2 - Les algorithmes classiques 18
3.2.1 - Chercher 18
3.2.2 - Trier 19
3.2.3 - Fusionner deux listes triées 19
3.3 - Les structures de données 19
3.3.1 - Les écrire à la main 19
3.3.2 - Celles du langage 19
3.3.3 - Rendre plusieurs valeurs : le p-uplet 20
3.4 - Tirer au hasard 20
3.5 - Les objets 20
3.5.1 - Premier pilier : l'encapsulation 21
3.5.2 - Deuxième pilier : l'héritage 21
3.5.3 - Troisième pilier : le polymorphisme 21
3.5.4 - Quatrième pilier : l'abstraction 21
3.6 - L'arbre binaire, écrit avec des objets 22
3.7 - Les graphes 22
3.8 - Écrire un résultat sur une ligne 22
3.9 - Géométrie algorithmique 23
3.9.1 - L'enveloppe convexe 23
3.9.2 - La paire la plus proche 23
3.9.3 - Triangulation de Delaunay et diagramme de Voronoï 24

1 - Collège

Version de démonstration : les saisies sont préremplies pour permettre la lecture intégrale. Les formulaires interactifs d'origine sont conservés dans archives.

1.1 - Ce que ce document couvre

Déclarer une variable et l'initialiser, poser une question au lecteur, décider selon une condition, répéter un traitement. Chaque structure s'y montre d'abord seule, puis les deux se mêlent : une condition à l'intérieur d'une boucle décide, tour par tour, ce qui reste dans le document.

1.2 - Variables et sortie

Une variable se pose avec `soit`, se relit avec `#`, entre dans un calcul avec `...` — l'équivalent d'une affectation suivie d'un affichage.

Le prix TTC est 180 euros.

1.3 - Une valeur initialisée

Pour conserver la navigation libre entre les chapitres, cet exemple utilise un entier déjà initialisé.

Pour découvrir la saisie interactive bloquante, consulter exclusivement l'atelier placé à la fin de basique.txt.

Tu es mineur(e).

1.4 - Une boucle simple

`pour` répète un bloc un nombre de fois connu à l'avance — l'équivalent d'un `for`.

Ligne 1 : le carré de 1 vaut 1.
Ligne 2 : le carré de 2 vaut 4.
Ligne 3 : le carré de 3 vaut 9.

1.5 - Une répétition sans compteur

`répéter ... fois` répète un bloc quand le rang n'a pas à paraître — la répétition du Logo, sans variable à nommer.

Une ligne de plus, à l'identique.
Une ligne de plus, à l'identique.
Une ligne de plus, à l'identique.

1.6 - Une condition simple

`si … sinon` choisit entre deux issues, une fois, sans répétition.

Admis avec 12.

1.7 - Une boucle avec un pas négatif

Le pas se règle en prose, comme tout réglage : `avec un pas de p`. Ici la boucle avance de 0,25 en 0,25.

Terme 1 : 2.
Terme 2 : 1,75.
Terme 3 : 1,5.
Terme 4 : 1,25.
Terme 5 : 1.
Terme 6 : 0,75.
Terme 7 : 0,5.
Terme 8 : 0,25.
Terme 9 : 0.

1.8 - Une boucle qui filtre

1.8.1 - Les multiples de 3 entre 1 et 20

Aucune ligne n'est écrite pour les tours silencieux — ils n'existent simplement pas dans le résultat. Chaque tour retenu forme sa propre ligne : c'est la boucle elle-même qui écrit le document, tour après tour.

n	Multiple de 3 ?
3	oui
6	oui
9	oui
12	oui
15	oui
18	oui

1.9 - Chercher les diviseurs d'un entier

Un entier d divise n lorsque le reste de la division de n par d est nul. Balayer tous les diviseurs candidats de 1 à n, et ne retenir que ceux qui divisent effectivement, donne la liste complète des diviseurs — la première brique, sur le papier comme ici, de tout test de primalité.

Diviseur	de 84
1	convient
2	convient
3	convient
4	convient
6	convient
7	convient
12	convient
14	convient
21	convient
28	convient
42	convient
84	convient

Un nombre premier n'a, par définition, aucun diviseur strictement compris entre 1 et lui-même. La table ci-dessus liste ceux de 84 — huit en tout, largement plus que les deux attendus d'un nombre premier.

1.10 - Filtrer par intervalle

`appartient à` et sa négation `n'appartient pas à` filtrent un intervalle aussi simplement qu'un test de divisibilité — utile chaque fois que l'algorithme cherche « à partir de quel rang » une grandeur franchit un seuil.

Rang	Terme	Écart
16	131071	65536
17	262143	131072
18	524287	262144
19	1048575	524288
20	2097151	1048576

1.11 - L'opérateur ternaire

`si ... sinon` peut aussi produire une valeur plutôt que du texte — affecté directement à une variable, comme un opérateur ternaire.

17 est impair.

1.12 - Conditions composées

`et` comme `ou` combinent deux conditions dans le même test ; les parenthèses en précisent l'ordre quand les deux se mêlent.

n	Retenu
15	multiple de 15
30	multiple de 15

Retenu par l'exception.

1.13 - L'accumulateur

Une variable posée avant la boucle et réaffectée à l'intérieur garde sa valeur d'un tour au suivant : c'est l'accumulateur, celui qui totalise une somme ou compte les cas retenus.

La somme des entiers de 1 à 100 vaut 5050.

La factorielle de 5 vaut 120.

1.14 - Compter sous condition

L'accumulateur placé à l'intérieur d'un `si` ne s'incrémente que sur les tours retenus — le compteur de l'algorithmique classique.

84 possède 12 diviseurs.

83 en possède 2 : c'est donc un nombre premier.

1.15 - Les boucles conditionnelles

`pour` répète un nombre de fois connu d'avance. Quand ce nombre dépend du calcul lui-même, c'est `tant que` qui convient : la condition est vérifiée **avant** chaque passage, si bien que la boucle peut n'en faire aucun.

La première puissance de 2 supérieure à 1000 est 1024, atteinte au rang 10.

La variante `faire … tant que` vérifie la condition **après** chaque passage : le corps s'exécute donc au moins une fois, même lorsque la condition est fausse dès le départ.

Passage unique, avec n valant 10.

Les deux formes reposent sur l'accumulateur de la section précédente : sans une variable qui avance d'un tour à l'autre, une boucle conditionnelle ne s'arrêterait jamais.

1.16 - Interrompre une répétition

Deux mots gouvernent le cours d'une boucle. `sortir` l'arrête entièrement — c'est la recherche qui cesse dès qu'elle a trouvé. `continuer` n'arrête que le tour en cours.

Le plus petit diviseur de 91 autre que 1 est 7.

[1]

[3]

[5]

Seuls les rangs impairs ont laissé une trace.

2 - Lycée

2.1 - Trois conteneurs, une seule grammaire

Un conteneur se déclare comme toute variable — le nom, le deux-points, le type en toutes lettres, puis la valeur. Le type est toujours écrit ; le point-virgule sépare les éléments ; l'accolade les rassemble. Les indices commencent à 0, comme en NSI.

2.2 - La liste

Le dièse affiche le conteneur entier : {12,5 ; 15 ; 9,5} — et l'indice lit un élément : la deuxième note est 15, le premier mot est chat. Un élément entre dans un calcul comme toute valeur : le double de la deuxième note vaut 30.

L'écriture indexée modifie une case existante, sans soit puisque rien n'est déclaré :

Après correction : {12,5 ; 15 ; 14}. L'indice 7 dépasserait les trois éléments disponibles : docdg refuserait cette lecture. Le cas fautif reste testé séparément.

2.2.1 - Grandir par concaténation

Une liste déclarée vide grandit par +, qui concatène toujours deux conteneurs — l'élément ajouté s'écrit donc en singleton, entre accolades. La réaffectation ne redit ni soit ni le type : ils ont été posés à la déclaration.

Les cinq premiers carrés : {1 ; 4 ; 9 ; 16 ; 25}.

2.3 - Le dictionnaire

La clé lit sa valeur : à pied, 5 minutes. Le + ajoute une paire :

Le dictionnaire complet : {Marche: 5 ; Bus: 3 ; Vélo: 7}. Chercher une clé absente comme Métro serait refusé : la démonstration utilise seulement les clés existantes.

La boucle dans livre les clés, et l'indexation retrouve chaque valeur :

Marche : 5 minutes.
Bus : 3 minutes.
Vélo : 7 minutes.

2.4 - La matrice

Une matrice porte ses dimensions dans son type. La forme littérale tient sur une ligne ; la forme bloc s'aligne à l'œil dans l'éditeur — tabulations entre colonnes, retours à la ligne entre rangées, aucun point-virgule :

A vaut $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ et B vaut $\begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Une case se lit par [ligne ; colonne] : 2 puis 7.

Déclarée avec {}, une matrice naît remplie de ses valeurs par défaut, puis se remplit case par case :

La table de multiplication : $\begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}$ — et sa case centrale : 4.

2.5 - Composer les conteneurs

Un tableau à plusieurs dimensions dont la taille n'est pas connue d'avance est une liste de listes. La rangée se déclare dans la boucle — elle repart vide à chaque tour — pendant que le conteneur extérieur, déclaré avant, accumule :

Le résultat : {{0 ; 0 ; 0 ; 0} ; {0 ; 1 ; 2 ; 3} ; {0 ; 2 ; 4 ; 6}}.

2.6 - Les quatre types de nombres

Le langage distingue les ensembles tels que les mathématiques les enseignent, et il respecte leur emboîtement $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{D} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$: un entier relatif appartient à $\mathbb{Z}$, un décimal à $\mathbb{D}$ — son écriture décimale est finie — un rationnel à $\mathbb{Q}$, un réel à $\mathbb{R}$, et un complexe à $\mathbb{C}$, sous sa forme algébrique $a + ib$.

Entiers : {1 ; -4 ; 12} — décimaux : {12,5 ; 0,25 ; 15} — réels : {1/3 ; racine(2) ; 2,5} — complexes : {1 + 2i ; -i}. Le second complexe vaut -i, dont la partie réelle est nulle.

L'inclusion $\mathbb{D} \subset \mathbb{Q}$ ne se remonte pas : un tiers est un rationnel parfaitement légitime, mais son écriture décimale est illimitée, donc il n'appartient pas à $\mathbb{D}$ — et docdg le dit, en nommant les deux ensembles et la sortie :

Une liste de décimaux ne peut pas contenir un tiers : le cas fautif, volontaire, reste dans les tests historiques.

2.7 - Le type protège

Le type déclaré est vérifié à la déclaration comme à la réaffectation. Une liste d'entiers contenant 2,5 serait refusée ; ce cas reste vérifié par les tests historiques.

2.8 - Mesurer, chercher, ranger

Un conteneur ne se contente pas de se remplir : il se mesure, se parcourt, se range. Le vocabulaire suit une règle simple — un nom pour ce qui produit une valeur, un verbe pour ce qui répond par oui ou non.

Écriture	Résultat
longueur(notes)	5
somme(notes)	65
min(notes)	9
max(notes)	18
trier(notes)	{9 ; 11 ; 12 ; 15 ; 18}
inverser(notes)	{11 ; 18 ; 9 ; 15 ; 12}
notes contient(15)	vrai
dans notes indice de(15)	1
dans notes insérer(0 ; 20)	{20 ; 12 ; 15 ; 9 ; 18 ; 11}
dans notes supprimer(2)	{12 ; 15 ; 18 ; 11}
dans notes ajouter(7)	{12 ; 15 ; 9 ; 18 ; 11 ; 7}
notes	{12 ; 15 ; 9 ; 18 ; 11}

Trois écritures pour une même opération, selon ce qui se lit le mieux : la forme fonctionnelle insérer(notes ; 0 ; 20), la forme prépositionnelle dans notes insérer(0 ; 20), et pour les questions par oui ou non la forme directe notes contient(15). C'est la phrase qui commande, non la machine.

2.8.1 - La division euclidienne

Le quotient et le reste se disent comme au collège, avec leurs prépositions :

Écriture	Résultat
quotient de 17 par 5	3
reste de 17 par 5	2
quotient de longueur(notes) par 2	2

La forme brève quotient(17 ; 5) reste admise, mais la phrase dit lequel des deux nombres est le dividende.

Chacune de ces écritures rend une **nouvelle liste : la série de départ n'est jamais modifiée. La primitive longueur mesure aussi bien un dictionnaire, une matrice ou une chaîne de caractères.**

Le conteneur peut aussi précéder la primitive, ce qui se lit comme une phrase : notes contient(15) rend vrai, et notes indice de(15) rend 1. Les deux écritures sont admises.

2.8.2 - *La chaîne de caractères*

Une chaîne se déclare comme un nombre — les guillemets délimitent la valeur sans lui appartenir.

liberté égalité fraternité compte 26 lettres, dont les huit premières sont liberté . La première est l — une lettre est une chaîne d'un seul caractère, docdg n'a pas de type distinct.

La chaîne se parcourt lettre à lettre. Les crochets ne servent qu'à rendre chaque tour visible : sans eux, sept lettres sur sept lignes consécutives ne feraient qu'un seul paragraphe, et se reliraient comme le mot de départ.

[e]
[x]
[a]
[c]
[t]

2.8.3 - *Écrire une suite sur une ligne*

Une boucle qui affiche met chaque tour sur sa ligne. Pour aligner une suite sur une seule ligne, avec le séparateur qu'on veut, on la compacte pour enlever les espaces, puis on la joint :

u, 11, c, o, d, e

12, 15, 9 — et avec un autre séparateur : 12 — 15 — 9.

Le chemin inverse découpe une ligne en ses morceaux : {un ; code}. Avec un séparateur vide, ce sont les lettres : {c ; o ; d ; e}.

Une chaîne écrite sur place se met entre guillemets. Sans eux, docdg refuse plutôt que de faire un tour unique sur la phrase entière.

Écriture	**Résultat**
majuscule("été")	ÉTÉ
minuscule("BONJOUR")	bonjour
sans accents("élève")	eleve
élaguer(" bonjour ")	[bonjour]
compacter("un code")	uncode
inverser(devise)	étrebil
devise contient("égalité")	vrai
code("B")	66
caractère(97)	a
texte(42)	42
nombre("1,5")	1,5

majuscule accentue les capitales — ÉTÉ et non ETE, comme l'exige la typographie française. code et caractère font le va-et-vient entre une lettre et son rang Unicode : c'est ce qui permet d'écrire un chiffrement.

Chercher un élément absent déclencherait une erreur explicite. Sur la liste existante, la position de 12 vaut 0.

2.8.4 - *La tranche*

Deux positions séparées par à désignent une portion, bornes incluses — comme pour n: un entier naturel de 1 à 5 fait cinq tours.

Les trois premières notes : {12 ; 15 ; 9}. La dernière seule : {11}.

Une tranche dont la borne de gauche dépasse celle de droite est simplement vide : {}. Ce n'est pas une faute, et c'est ce qui permet de conclure une fusion de deux listes triées en une seule ligne.

2.8.5 - *Le classement des chaînes*

{avion ; école ; èdre ; Zoé}

L'accent ne retourne pas le mot à la fin de l'alphabet : la comparaison porte d'abord sur les lettres dépouillées, l'accent ne servant qu'à départager — comme dans un dictionnaire.

2.9 - Deux sortes de fonctions

docdg distingue la fonction mathématique — objet d'étude qu'on dérive, qu'on trace, dont on cherche les limites — et la fonction algorithmique, calcul nommé qu'on réutilise. Les chevrons marquent la première, le soit nu la seconde, exactement comme ils distinguent déjà la matrice de l'algèbre du conteneur de l'algorithmique.

Une fonction algorithmique déclare le type de chacun de ses paramètres et celui de sa valeur, avec l'article, comme partout ailleurs dans le langage : un entier, une liste de réels. Elle s'appelle ensuite par son nom, arguments séparés par le point-virgule comme toute donnée.

2.10 - La forme la plus simple

L'appel se fait dans un calcul : 10, ou 42.

Les types ne sont pas décoratifs : un décimal à la place d'un entier, ou un argument manquant, seraient refusés. Avec deux entiers, le calcul est valide : 9.

2.11 - Un corps entre accolades

Quand le calcul tient en une expression, l'accolade est facultative :

L'aire d'un rectangle de 2,5 sur 4 vaut 10.

2.11.1 - Plusieurs instructions

Dès que le corps compte plusieurs lignes, la valeur produite doit être désignée par retourne — rien n'est implicite :

Le triangle de côtés 3 et 4 a pour hypoténuse 5, celui de côtés 5 et 12 a pour hypoténuse 13.

Une fonction dont le corps ne retourne rien serait refusée. Voici la forme correcte avec un retour explicite :

Appel : 3.

2.12 - Fonctions et boucles

Une fonction s'appelle partout où un calcul est admis — dans le texte, dans une rangée de tableau, dans la condition d'une boucle, dans un accumulateur.

Les six premiers carrés, calculés par la fonction et rassemblés dans une liste : {1 ; 4 ; 9 ; 16 ; 25 ; 36}.
Et le test de divisibilité, écrit une fois, réutilisé partout :

Diviseur	de 60
1	convient
2	convient
3	convient
4	convient
5	convient
6	convient
10	convient
12	convient
15	convient
20	convient
30	convient
60	convient

2.13 - Recevoir et rendre un conteneur

Un paramètre peut être une collection, un dictionnaire, une matrice ou un texte : son type s'écrit comme celui d'une variable.
La moyenne de la série vaut 13,2.
La primitive longueur mesure un conteneur — collection, dictionnaire, matrice ou texte.
Une collection d'entiers est acceptée là où une liste de réels est attendue : la fonction reçoit une copie, elle ne peut donc rien renvoyer dans la série de l'appelant.

2.13.1 - Rendre une collection

La série {1 ; 2 ; 3} doublée donne {1 ; 2 ; 3 ; 1 ; 2 ; 3}.

2.13.2 - Chercher dans une collection reçue

15 occupe la position 2 ; 7 n'y figure pas : -1.

2.13.3 - Modifier une copie locale

Un nom posé sur un conteneur reçu en désigne une copie : l'écriture indexée n'atteint jamais la série de l'appelant.

Copie modifiée : {100 ; 2 ; 3} — série d'origine : {1 ; 2 ; 3}.

2.13.4 - Les chaînes de caractères

bonjour

Les types restent vérifiés : passer un nombre seul à la place d'une liste serait refusé. Avec une véritable liste, la recherche de 3 renvoie 2.

2.14 - Les primitives dans un corps

Les primitives de conteneur — longueur, trier, inverser, somme, min, max, quotient, contient, indice de, insérer, supprimer, ajouter — s'emploient partout, y compris dans un corps de fonction :

La médiane de {7 ; 1 ; 5 ; 3 ; 9} vaut 5 — la série est d'abord triée, puis on lit la valeur du milieu. La formule vaut pour un nombre impair de valeurs.

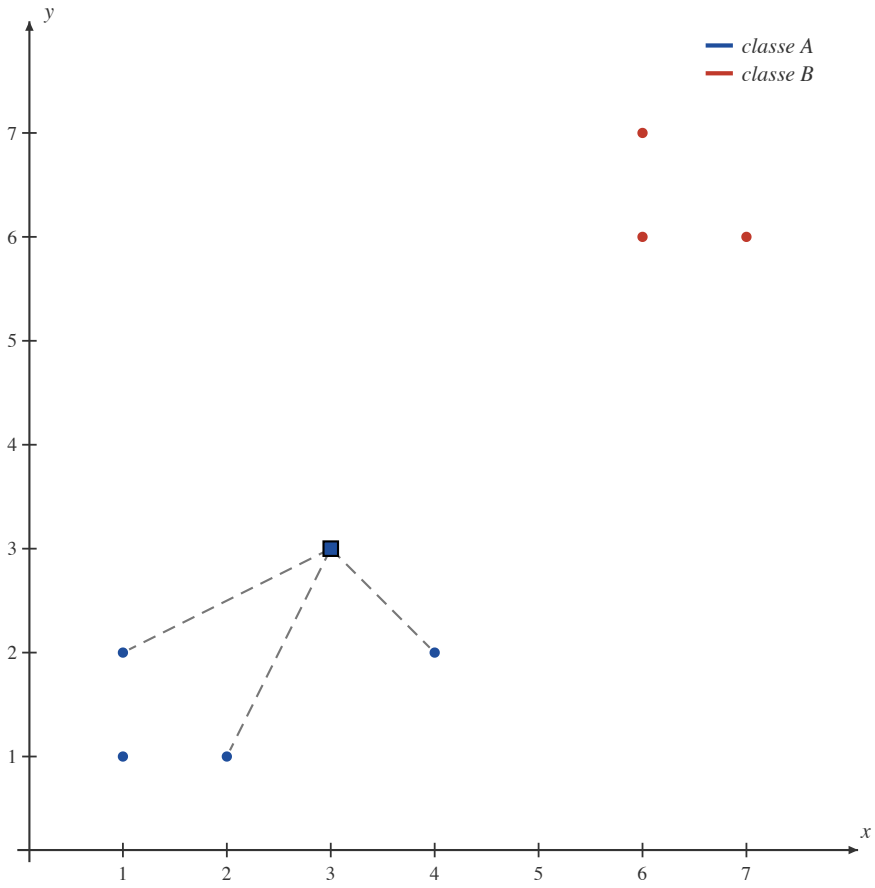
Un nom de fonction tient en un seul mot : le nom composé est réservé aux mots du langage lui-même, comme indice de ou sans accents. Quand deux mots sont vraiment nécessaires — parce que l'algorithme porte ce nom, tel le tri par insertion — le trait bas les relie : tri_par_insertion. Partout ailleurs, un mot suffit et se lit mieux.

2.15 - Les k plus proches voisins

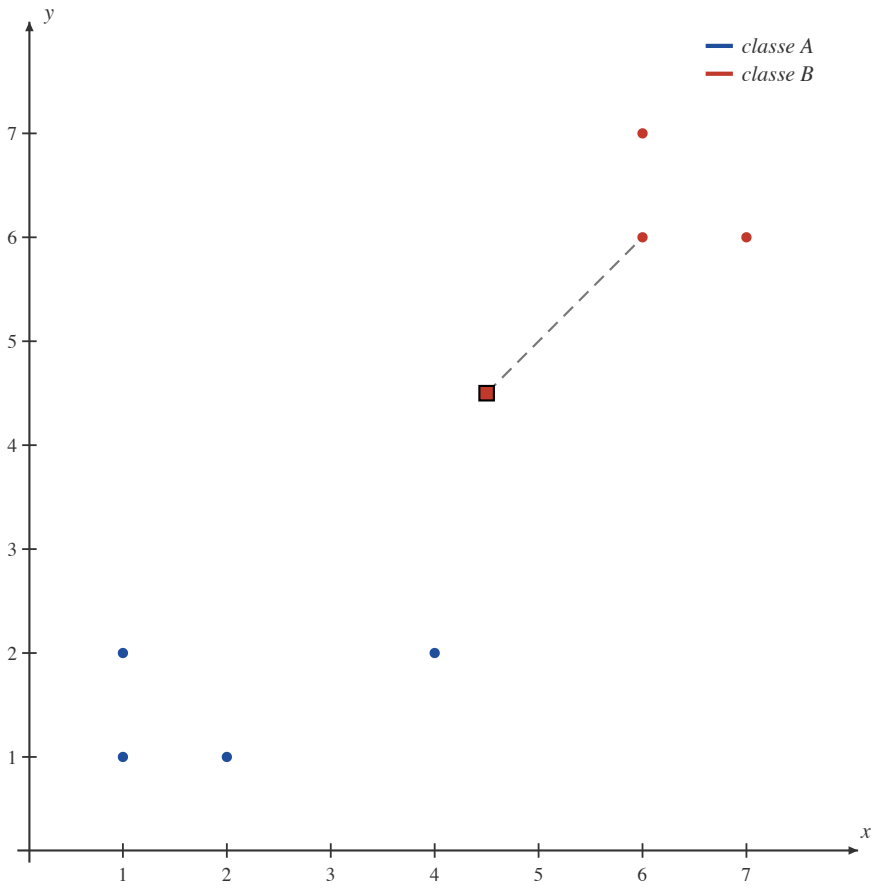
Pour classer un nouveau point, on regarde la classe de ses k plus proches voisins dans un jeu d'apprentissage, et l'on vote : c'est l'algorithme des k plus proches voisins, première méthode d'apprentissage automatique du programme. docdg cherche les voisins par un arbre k-d et affiche le vote.

Les 3 plus proches voisins de $(3\,;\,3)$, trouvés par un arbre k-d : $P_7(4\,;\,2)$ de classe A, à distance $\approx 1{,}414$; $P_2(2\,;\,1)$ de classe A, à distance $\approx 2{,}236$; $P_3(1\,;\,2)$ de classe A, à distance $\approx 2{,}236$.

Vote (A : 3) : le point est classé dans la classe A.



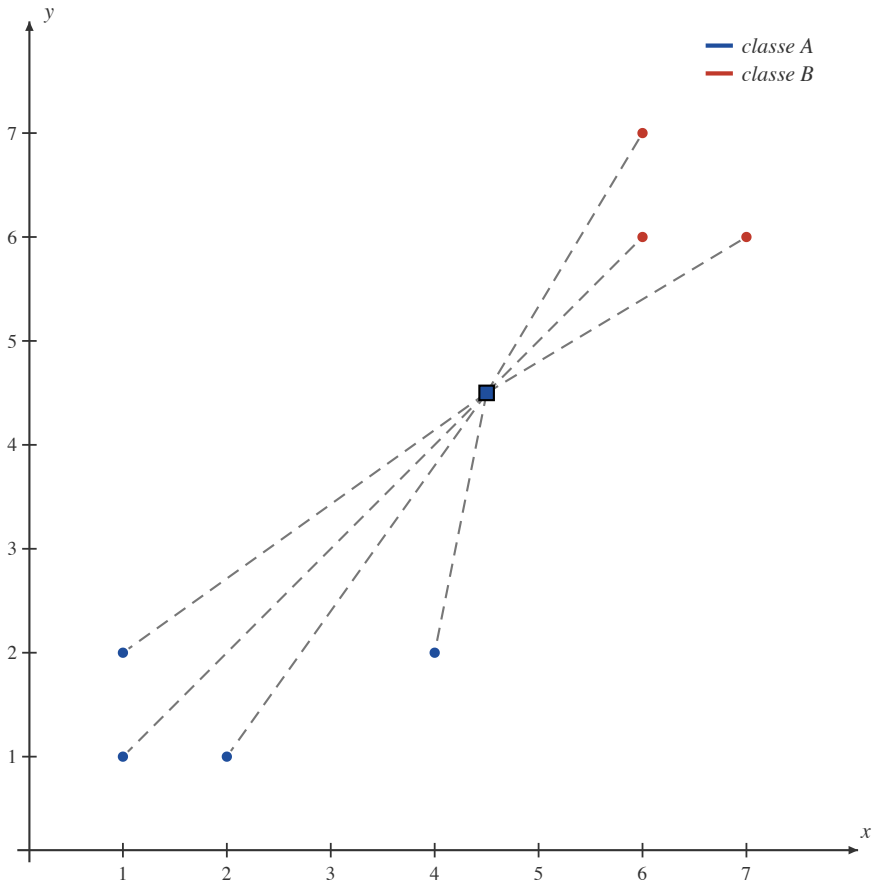
Le choix de k compte. Pour le point (4,5 ; 4,5), le plus proche voisin est de classe B :
Les 1 plus proches voisins de $(4{,}5\ ;\ 4{,}5)$, trouvés par un arbre k-d : $P_4(6\ ;\ 6)$ de classe B, à distance $\approx 2{,}121$.
Vote (B : 1) : le point est classé dans la classe B.



Avec k égal à la taille du jeu, le vote se réduit à la classe majoritaire, quel que soit le point : k trop grand noie l'information locale.

Les 7 plus proches voisins de $(4{,}5\,;\,4{,}5)$, trouvés par un arbre k-d : $P_4(6\,;\,6)$ de classe B, à distance $\approx 2{,}121$; $P_7(4\,;\,2)$ de classe A, à distance $\approx 2{,}55$; $P_6(6\,;\,7)$ de classe B, à distance $\approx 2{,}915$; $P_5(7\,;\,6)$ de classe B, à distance $\approx 2{,}915$; $P_3(1\,;\,2)$ de classe A, à distance $\approx 4{,}301$; $P_2(2\,;\,1)$ de classe A, à distance $\approx 4{,}301$; $P_1(1\,;\,1)$ de classe A, à distance $\approx 4{,}95$.

Vote (A : 4, B : 3) : le point est classé dans la classe A.



3 - Enseignement supérieur

3.1 - La récursivité

Une fonction peut s'appeler elle-même. La condition d'arrêt s'écrit avec si … sinon, qui produit ici une valeur plutôt qu'un texte :

Les premières factorielles : 1, 2, 6, 24, 120, 720.

La suite de Fibonacci se définit de la même façon, avec deux appels par tour :

Les douze premiers termes, dressés par une boucle qui appelle la fonction à chaque rang :

Rang	Terme
0	0
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21
9	34
10	55
11	89

La récursion est bornée : au-delà de deux cents appels imbriqués, docdg s'arrête et le dit, plutôt que d'épuiser la pile.

3.2 - Les algorithmes classiques

Les exercices du programme s'écrivent avec ce qui précède : une fonction, une boucle, une collection reçue en copie.

3.2.1 - Chercher

Le `retourne` placé dans la boucle en sort sur-le-champ : c'est le geste que montre tout manuel. Dans une série triée, la dichotomie coupe l'intervalle en deux à chaque tour.

11 occupe la position 4 ; recherche(premiers ; 4) ne le trouve pas : -1.

3.2.2 - Trier

Méthode	Résultat
Insertion	{1 ; 2 ; 5 ; 5 ; 6 ; 9}
Sélection	{1 ; 2 ; 5 ; 5 ; 6 ; 9}
Série d'origine	{5 ; 2 ; 9 ; 1 ; 5 ; 6}

La série de départ n'a pas bougé : une fonction rend une valeur, elle ne modifie rien à distance. La primitive trier(désordre) donne le même résultat — {1 ; 2 ; 5 ; 5 ; 6 ; 9} — mais on n'apprend pas à trier en l'appelant.

3.2.3 - Fusionner deux listes triées

{1 ; 2 ; 3 ; 4 ; 7 ; 8 ; 9}

À la sortie de la boucle, l'une des deux tranches finales est toujours vide — une tranche dont la borne de gauche dépasse celle de droite n'est pas une faute. C'est ce qui permet de conclure en une ligne.

3.3 - Les structures de données

3.3.1 - Les écrire à la main

Sept lignes suffisent, et aucune n'est du moteur : la structure du programme de terminale se démontre au tableau au lieu d'être cachée dans le logiciel. On les nomme ici avec un suffixe, car une fonction qu'on écrit masque la primitive du même nom — c'est voulu, pour que l'élève puisse écrire la sienne, mais cela empêcherait d'employer les deux dans un même document.

La pile {1 ; 2 ; 3} a pour sommet 3 ; après dépilement il reste {1 ; 2}. La file {1 ; 2 ; 3} a pour tête 1 et laisse {2 ; 3}.

Le type de p s'écrit comme partout ailleurs : une seule règle, sans exception pour les valeurs composées.

3.3.2 - Celles du langage

Une fois l'exercice fait, le langage les fournit — avec leur discipline, qui est tout leur intérêt. Une file n'a pas de sommet, une pile n'a pas de tête, et aucune des deux ne s'indexe.

La pile {1 ; 2} a pour sommet 2 ; dépilée, il reste {1}. Elle est vide ? faux

La file {1 ; 2 ; 3} a pour tête 1 ; défilée, il reste {2 ; 3}.

3.3.3 - Rendre plusieurs valeurs : le p-uplet

Un p-uplet a une longueur fixe et des types qui peuvent différer, là où une collection est homogène et de longueur libre. Il s'écrit entre parenthèses, comme un point du plan.

17 = 5 × 3 + 2 — la déliaison pose les deux noms d'un coup.

L'exercice qui justifie à lui seul le p-uplet : deux résultats, un seul parcours.

La plus petite valeur est 9, la plus grande 18.

Les membres se lisent aussi par leur rang, et les types peuvent différer :

(3 ; trois) — dont le second membre est trois.

3.4 - Tirer au hasard

aléatoire(a ; b) rend un entier entre les deux bornes, comprises. Deux compilations donnent deux tirages : c'est ce qu'une simulation attend.

Sur 60 lancers d'un dé, 10 six — pour une espérance de 10.

3.5 - Les objets

Une classe rassemble des données et ce qu'on sait en faire. Elle se déclare comme une phrase de mathématiques, et le type de ses attributs porte son article comme partout ailleurs : une seule règle, du premier `soit` au dernier.

Le point Point(abscisse: 3 ; ordonnée: 4) a pour distance à l'origine 5. Translaté d'une unité dans chaque direction, il devient Point(abscisse: 4 ; ordonnée: 5).

Le nom d'une classe commence par une majuscule : c'est ce qui le distingue d'un type du langage. Un attribut se lit avec un point, une méthode s'appelle de même. Dans le corps d'une méthode, les attributs sont visibles par leur nom, sans rien devant : la formule s'écrit comme au tableau.

Un attribut se modifie sans soit, puisque rien n'est déclaré :

Le point q vaut Point(abscisse: 6 ; ordonnée: 2).

3.5.1 - Premier pilier : l'encapsulation

Une classe peut garder pour elle ce qui ne regarde pas le dehors. Un seul mot le dit, et tout ce qui n'est pas marqué reste visible.

Le titulaire se lit du dehors : Léa. Les intérêts se calculent : 5 — la méthode publique lit l'attribut privé et appelle la méthode privée, ce qui est tout l'objet de l'encapsulation.

Le solde ne se lit pas du dehors, et la méthode privée taux ne peut pas être appelée de l'extérieur : ces deux tentatives sont vérifiées dans les tests historiques.

3.5.2 - Deuxième pilier : l'héritage

Une classe peut reprendre une autre et la prolonger. Elle en reçoit les attributs, les méthodes et les secrets.

Rex reçoit de sa classe mère la méthode carte, qui donne Rex, et redéfinit cri, qui donne ouaf. L'animal, lui, garde le sien :

…

3.5.3 - Troisième pilier : le polymorphisme

Un chien tient la place d'un animal. C'est la valeur qui décide de la méthode appelée, non le type écrit :

ouaf — la fonction attend un animal, elle a reçu un chien, et c'est le cri du chien qui sort.

Rangés ensemble, chacun garde le sien :

[ouaf]
[miaou]

3.5.4 - Quatrième pilier : l'abstraction

Une classe peut dire ce que ses filles doivent savoir faire, sans dire comment. Une méthode déclarée sans membre droit n'a pas de corps.

Un carré de 3 a pour aire 9 ; un rectangle 2 sur 5 a pour aire 10. La fonction mesure ne sait rien de l'une ni de l'autre : elle sait seulement qu'une forme a une aire.

Une classe abstraite ne s'instancie pas : seules les classes concrètes Carré et Rectangle peuvent créer des objets. Le refus de l'instanciation directe reste testé séparément.

3.6 - L'arbre binaire, écrit avec des objets

Les quatre piliers suffisent à écrire un arbre, sans avoir besoin d'une valeur nulle : l'arbre vide est une classe. La hiérarchie dit tout — un arbre est vide, ou c'est un nœud.

Cet arbre compte 3 nœuds et sa hauteur vaut 2.

Un attribut peut donc contenir un objet : c'est ce qui permet à une structure de se contenir elle-même.

3.7 - Les graphes

Un graphe se donne par sa liste d'adjacence : un dictionnaire dont chaque clé est un sommet et chaque valeur la liste de ses voisins.

Le graphe compte 4 sommets ; les voisins de A sont {B ; C}.

Les deux parcours diffèrent par une seule chose : la file donne le parcours en largeur, la pile le parcours en profondeur.

Depuis A, en largeur : ABCD. En profondeur : ACDB.

3.8 - Écrire un résultat sur une ligne

Les nombres premiers jusqu'à 50 : 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47.

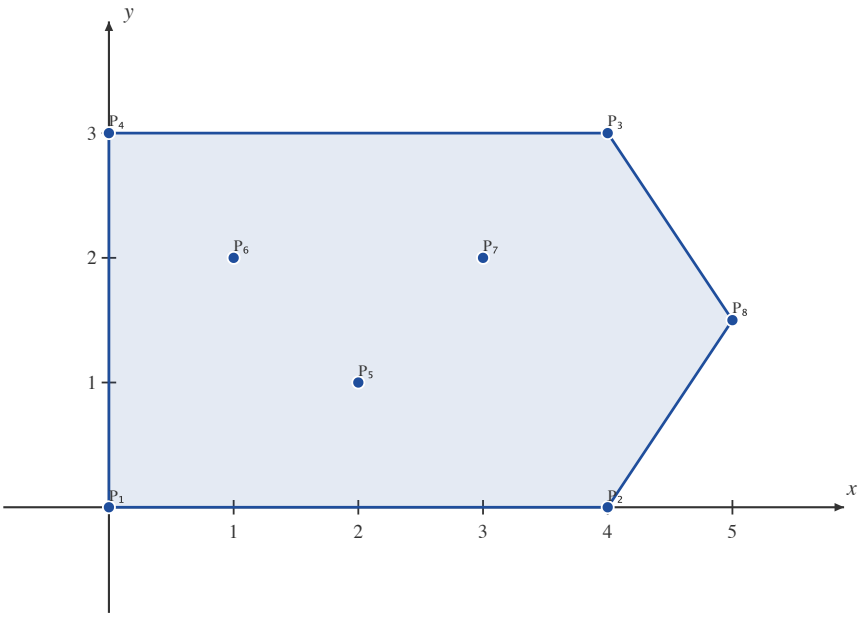
La primitive joindre évite l'accumulateur, qui laisse toujours un séparateur de trop à la fin.

3.9 - Géométrie algorithmique

Les algorithmes classiques de la géométrie algorithmique du plan : l'enveloppe convexe (le plus petit polygone convexe contenant les points, que l'on construit à la main par le parcours de Graham), la paire de points la plus proche, la triangulation de Delaunay et son dual, le diagramme de Voronoï.

3.9.1 - L'enveloppe convexe

Enveloppe convexe des 8 points (algorithme Quickhull) : 5 sommets, P_8, P_3, P_4, P_1, P_2 dans le sens direct ; aire $13{,}5$, périmètre $14{,}61$.



3.9.2 - La paire la plus proche

L'algorithme naïf compare toutes les paires, soit $n(n-1)/2$ comparaisons ; la méthode « diviser pour régner » descend à $n \log n$.

Les deux points les plus proches sont $P_3(2\,;\,4)$ et $P_5(2{,}5\,;\,4{,}5)$, à distance $\approx 0{,}7071$ — trouvés parmi les 15 paires.

3.9.3 - Triangulation de Delaunay et diagramme de Voronoï

Pour n points dont h forment l'enveloppe convexe, la triangulation de Delaunay compte exactement 2n − h − 2 triangles. Les sommets du diagramme de Voronoï sont les centres des cercles circonscrits de ses triangles.
Triangulation de Delaunay des 7 points : 8 triangles, dont aucun cercle circonscrit ne contient d'autre point ; elle maximise le plus petit angle, ce qui en fait le maillage de référence.

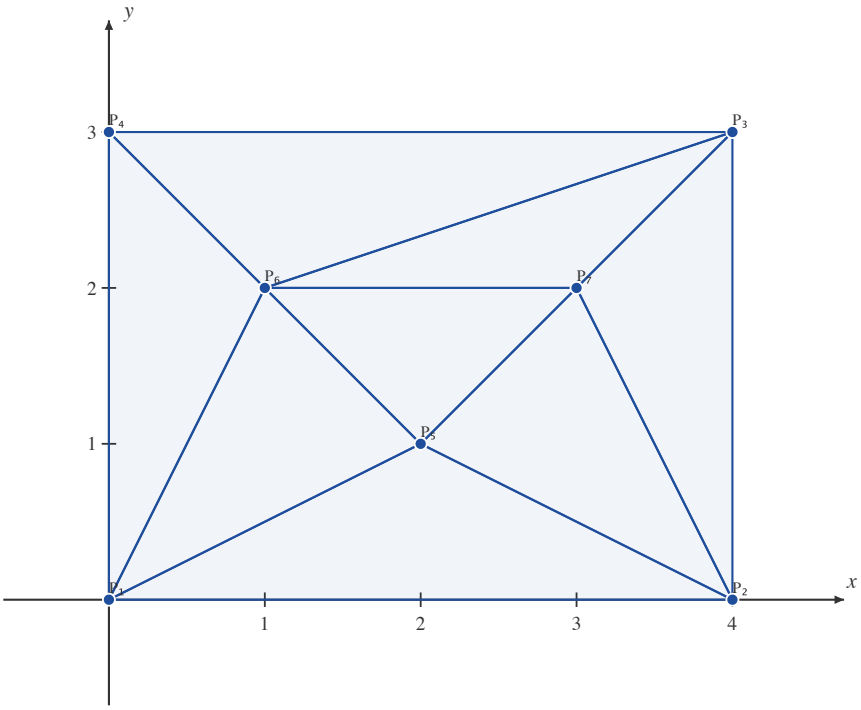


Diagramme de Voronoï des 7 germes : chaque cellule rassemble les points du plan plus proches de son germe que de tout autre ; les cellules du bord, infinies, sont tronquées au cadre.

